

Tutorial

Setting up the environment:

Prerequisites:

- NVIDIA GPU with drivers already installed. Confirm with the command `nvidia-smi`
- The uv package installed. If not installed, run the command `curl -LsSf https://astral.sh/uv/install/sh | sh`

Instructions:

1: Create the virtual environment. Note that for this project I used Python 3.12.13

```
mkdir -p ~/envs
```

```
cd ~/envs
```

```
uv venv --python 3.12 yolo26
```

2: Activate the virtual environment. Once you run this command, `(yolo26)` should appear in front of the terminal prompt

Shell

```
source ~/envs/yolo26/bin/activate
```

3: Install PyTorch with CUDA support separately. This needs to be separate from the later requirements.txt install because it needs a specific URL matching the version of CUDA you have.

Shell

```
uv pip install torch torchvision --index-url  
https://download.pytorch.org/whl/cu130
```

4: Install all the libraries that the scripts I wrote for the project need. Use uv pip, not pip, to do this:

Shell

```
uv pip install ultralytics opencv-python numpy pyyaml  
matplotlib pandas albumentations
```

The libraries are:

- Ultralytics: Library from which YOLO26 pre-trained models can be imported
- OpenCV-Python: Extensive open-source computer vision library. This project uses it for contour extraction and mask reading to convert masks to the format YOLO expects
- NumPy: Extensive library for array and matrix-vector operations used throughout the project.
- Pyyaml: Used to read and write data.yaml and args.yaml, configuration files that direct training by pointing to the folders containing training and testing data.
- Matplotlib: Library for making scientific plots and graphs. Used to create plots of loss and performance vs epoch, IoU histograms, etc.
- Pandas: Library for reading and manipulating spreadsheets or csvs of data. Used to parse YOLO's results.csv file to create loss plots.
- Albumentations: Library with custom, additional data augmentation techniques. Used to introduce an optional augmentation that adds Gaussian noise to data.

5: Run Ultralytics's check to ensure all the installs required for YOLO are present, and the right versions of each install are present.

Shell

```
yolo checks
```

6: Freeze the environment and create a requirements.txt folder.

Shell

```
cd ~/envs/yolo26  
uv pip freeze > requirements.txt
```

7: Install tmux, which is used to survive terminal disconnection, very useful for long training runs.

Shell

```
sudo apt install tmux
```

8: If you wish to recreate this environment on a new machine, run the following:

Shell

```
uv venv --python 3.12 yolo26
source yolo26/bin/activate
uv pip install torch torchvision --index-url
https://download.pytorch.org/whl/cuXXX
uv pip install -r requirements.txt
```

Dataset Setup, Training & Evaluation

Activate the virtual environment before running anything (command above).

convert_cme_dataset.py

The raw synthetic images and masks are found in the following folder structure:

~/synthetic_images/cme_seg_20250320_full_size/

```
|— 11329/
|   |— <params>_sat2_bt看.png    <- the training image
|   |— <params>_sat2_back.png    <- unused
|   |— mask/
|       |— <params>_sat2_mask_0.png <- OCCULTER binary mask
|       |— <params>_sat2_mask_1.png <- unused
|       |— <params>_sat2_mask_2.png <- CME binary mask
|       |— <params>_sat2_mask_3.png <- unused
|— 11330/
```

YOLO26 cannot directly train on this layout; it requires a standard structure where data is sorted into folders called images/ (for training images) and labels/ (for masks). Additionally, YOLO requires masks to be in a list format (list of pixel coordinates), rather than a 2D binary array. In addition, data needs to be split into training and validation datasets in a random manner, and a

data.yaml file that specifies the folder location of the training vs validation datasets and images vs labels needs to be created. All this is handled by running **convert_cme_dataset.py**:

python ~/convert_cme_dataset.py

Note for reproducibility that `random.seed(42)` is used to generate the random train-test split.

The output, in the correct format for YOLO, looks like this:

~/datasets/cme_yolo_seg_100/

```
|— images/{train,val}/  
|— labels/{train,val}/  
|— data.yaml
```

Training

My project trains using the file **train_with_tracking.py**. This file wraps `ultralytics.YOLO.train()` with project-specific extra functionality, such as the ability to calculate certain segmentation metrics on the validation dataset after every epoch of training.

Important flags:

```
--data PATH          data.yaml (required)  
--original PATH      raw source dataset, for GT masks used by the eval callback  
                    (defaults to the Cartesian full-size source)  
--model NAME         e.g. yolo26n-seg.pt / yolo26s-seg.pt / yolo26m-seg.pt  
--epochs N  
--imgsz N            default 512  
--batch N  
--device N  
--name NAME          run folder name under --project (default ~/runs/cme_yolo)  
--eval_every N       how often the custom per-epoch IoU/PQ callback fires  
--workers N          dataloader worker count — IMPORTANT, see note below  
--disable_builtin_ augmentations (should ALWAYS be enabled to prevent Ultralytics's automatic  
augmentations)
```

Extra augmentation flags (all other augmentation is off by default as long as

`--disable_builtin_ augmentations` is enabled)

`--hsv_h N` (color hue shift)

`--degrees N` (rotation by N degrees)

`--fliplr` (flip the image along vertical axis)

--flipud (flip the image along horizontal axis)
--gaussian_noise_std (standard deviation for Gaussian noise)
--gaussian_noise_p (mean for Gaussian noise)

Other flags:

--polar (triggers training with a dataset of polar rather than Cartesian images; more info below)

Example of the training run launch pattern used throughout this project:

1: Check system health

```
ps aux | grep train_with_tracking | grep -v grep  
free -h  
df -h ~
```

2: Launch inside tmux

```
tmux new -s my_training_run
```

```
~/envs/yolo26/bin/python ~/train_with_tracking.py \  
--data ~/datasets/cme_yolo_seg_100/data.yaml \  
--model yolo26s-seg.pt \  
--name "$NAME" \  
--epochs 50 --imgsz 512 --batch 32 --device 0 --eval_every 10 \  
--workers 4 \  
--disable_builtin_augmentations
```

Polar transformation

Optionally, you can transform every image and mask from Cartesian to polar before training; this is done with functions in **polar_utils.py** (`get_polar_params()`, which returns the image center and max radius to be passed into `to_polar()`, which uses a mathematical function to convert Cartesian images to polar. The functions in this file are used by **create_polar_dataset.py**, which creates a separate dataset for polar images, by setting up data in the same manner as **convert_cme_dataset.py** but also applying polar transformation to images and masks. These files are not called directly; rather, they are called if the flag `--polar` is included when running `train_with_tracking.py`.

Evaluation

To evaluate run the following files:

eval_final.py outputs the following:

- per_image_metrics.csv (csv file displaying metrics like IoU, PQ, F1 corresponding to each image in the validation dataset)
- pred_masks/ (saved binary predicted mask PNGs)
- iou_hist_cme.png (histogram of CME IoU across the validation dataset)

To run **eval_final.py** run the following command:

None

```
cd ~/runs/cme_yolo/<run_name>
~/envs/yolo26/bin/python ~/eval_final.py \
  --model weights/best.pt \
  --data ~/datasets/cme_yolo_seg_100/data.yaml \
  --outdir eval_outputs \
  | tee eval_outputs/pq_summary.txt
```

plot_loss.py produces the following loss curves:

- train_losses.png
- val_losses.png

To run **plot_loss.py** run the following command:

None

```
~/envs/yolo26/bin/python ~/plot_loss.py --run_dir .
```

plot_epoch_curves.py and **plot_combined.py** produce the following files:

- iou_over_epochs.png (graph of mean, median IoU against number of training epochs)

-pq_over_epochs.png (graph of mean, median Panoptic Quality against number of training epochs)
-combined_loss_iou.png (combined graph showing training and validation loss, 1st quartile IoU, median IoU, and 3rd quartile IoU against number of training epochs)

To produce these files run the following commands:

None

```
~/envs/yolo26/bin/python ~/plot_epoch_curves.py --run_dir .  
~/envs/yolo26/bin/python ~/plot_combined.py --run_dir .
```

eval_threshold.py produces iou_vs_mask_threshold.png, a graph of the pixel-probability cutoff threshold used by YOLO26 vs median, 1st quartile, and 3rd quartile IoU, used to determine the optimal probability cutoff threshold that should be chosen. To run this file, run the following command:

None

```
~/envs/yolo26/bin/python ~/eval_threshold.py \  
  --model weights/best.pt \  
  --data ~/datasets/cme_yolo_seg_100/data.yaml \  
  --outdir eval_outputs
```

eval_examples.py

Running this file produces:

-iou_bin_overview.png: one visual example of model output masks vs. ground truth masks per IoU bin (0.5-1.0, in 0.05 steps, so for example one example ~0.55 IoU, one example ~0.6 IoU, etc)

-iou_bin_overview_contours.png (CME probability contour maps corresponding to the images used as examples in iou_bin_overview.png)

-worst_{n}_examples.png (n is a parameter passed in when running the file, with the flag --n_worst N. This image shows examples of model output masks vs. ground truth masks where the model fails, by selecting images where the model fails to make a correct prediction and images where the model makes a correct prediction but the IoU is extremely low)

To run, run the following command:

None

```
~/envs/yolo26/bin/python ~/eval_examples.py \  
  --run_dir . \  
  --model weights/best.pt \  
  --n_worst 20
```

eval_real_obs_contours.py

Run this file to evaluate the model's performance in generalization to real CME images, if a folder of such images is available. This file produces two outputs per real image: an image showing the model's predicted classes and masks overlaying the image, and an image showing CME probability contours for each image.

To run, run the following. Use --img_start and --img_end if you only wish to evaluate specific images within the folder (e.g. the 69th to 99th images if using the real_obs_SPD dataset)

None

```
~/envs/yolo26/bin/python ~/eval_real_obs_contours.py \  
\  
  --model ~/runs/cme_yolo/<run_name>/weights/best.pt \  
\  
  --model_name <name> \  
  --image_dir ~/<image_dir_name> \  
  --outdir ~/<image_dir_name>/predictions_contours \  
  --img_start N --img_end N
```